
BA-Sauerteig

Release 1.0

Quyiequeg

16.06.2026

Inhaltsverzeichnis

Python-Modulindex

15

`cost.cost_function_whole(ordering: list[int], node_grps: dict, edges: list) → int`

Die Kostenfunktion wird in Schritt zwei der Pipeline (Sec. 3.2) verwendet, um die kostengünstigste Achsenordnung zu ermitteln. Kostengünstig bedeutet hier, dass möglichst viele Achsenpaare mit möglichst geringem Span einfließen sollen.

Parameter

- **ordering** (*list[int]*) – Ordnung der Achsen (ϕ)
- **node_grps** (*dict*) – key: Achse, value: Knoten auf der Achse
- **edges** (*list*) – Kantenliste des Graphen

Rückgabe

Gesamtkosten der Achsenordnung

Rückgabetyp

int

`cost.edges_between_axes(node_grps, edges, a1, a2)`

Ermittelt wieviele Kanten zwischen den gegebenen Achsen bestehen.

Parameter

- **node_grps** (*dict*) – key: subset, value: list
- **edges** (*list*) – Kantenliste aus Tupeln
- **a1** (*int*) – Achse 1
- **a2** (*int*) – Achse 2

Rückgabe

Anzahl der Kanten

Rückgabetyp

int

`cost.node_or_axes_span(n1, n2, k)`

Berechnet den Spann zweier Knoten oder Achsen

Parameter

- **n1** (*int*) – Achse/Knoten 1
- **n2** (*int*) – Achse/Knoten 2
- **k** (*int*) – Gesamtzahl der Achsen

Rückgabe

errechneter Spann

Rückgabetyp

int

`crossing_minimization.barycenter_crossmin_pipeline(layout: HivePlotLayout, threshold: float = inf, expanded: bool = False) → None`

Führt die Kreuzungsminimierungs-Pipeline mit der Barycenterheuristik aus (3a/3b).

Die Pipeline umfasst im expandierten Fall: 1. anfängliches Layout expandieren 2. Entfernen isolierter Knoten auf den Achsen. 3. Einfügen von Dummy-Knoten für lange Kanten (Span > 1) und sortieren der inter axis Knoten (Span = 0). 4. Initialisierung des Sweeps und Updates der Datenstrukturen. 5. Mehrfache CW/CCW-Sweeps für reale und anschließend virtuelle Knoten, die die Barycenterpositionen ermitteln und die Achsen entsprechend sortieren. 6. Herstellen der Achsenordnung und letztes Update der Datenstrukturen um die Visualisierung starten zu können.

Die Pipeline umfasst im nicht expandierten Fall: 1. Entfernen isolierter Knoten auf den Achsen. 2. Einfügen von Dummy-Knoten für lange Kanten (Span > 1) und sortieren der inter axis Knoten (Span = 0). 3. Initialisierung des Sweeps und Updates der Datenstrukturen. 4. Mehrfache CW/CCW-Sweeps für reale und

anschließend virtuelle Knoten, die die Barycenterpositionen ermitteln und die Achsen entsprechend sortieren. 5. Sortierung der reinen intra axis Knoten. 6. Herstellen der Achsenordnung und letztes Update der Datenstrukturen um die Visualisierung starten zu können.

Parameter

- **layout** ([HivePlotLayout](#)) – Das zu optimierende HivePlotLayout.
- **threshold** (*int*) – Optionaler Abbruchschwellwert für die Anzahl der Sweep-Durchläufe.
- **expanded** (*bool*) – dient der Unterscheidung, ob in der Pipeline mit expandierten Achsen gerechnet wird oder nicht, Default = False (nicht expandierter Fall)

`crossing_minimization.calculate_barycenter_position(layout: HivePlotLayout, neighbor_group: list[int], node_axis_map, expanded: bool = False) → float`

Berechnet die Barycenterposition eines Knotens über seine ermittelten Nachbarn. Siehe `HivePlotLayout.get_proper_neighbors()`.

Parameter

- **layout** ([HivePlotLayout](#)) – das zu berechnende HivePlotLayout
- **neighbor_group** (*list[int]*) – Liste der Nachbarknoten
- **expanded** (*bool*) – dient der Unterscheidung, ob in der Pipeline mit expandierten Achsen gerechnet wird oder nicht, Default = False (nicht expandierter Fall)

Rückgabe

Barycenterposition

Rückgabetyp

float

`crossing_minimization.edge_direction(start_pos: int, end_pos: int, k: int) → int`

Ermittelt die Richtung einer Kante in Form der nächsten Achse, die die Kante überquert. Diese spiegelt sowohl die Richtung im Uhrzeigersinn oder gegen den Uhrzeigersinn als auch die Position des ersten Dummyknotens wieder, sollte es sich um eine lange Kante handeln. Für `Span <= 1` wird simpel der Endknoten zurückgegeben.

Parameter

- **start_pos** (*int*) – Achsenposition in `hpl.node_groups` des Startknotens
- **end_pos** (*int*) – Achsenposition in `hpl.node_groups` des Endknotens
- **k** (*int*) – Anzahl der Achsen

Rückgabe

Achsenposition in `hpl.node_groups` in Richtung der langen Kante

Rückgabetyp

int

`crossing_minimization.edge_node_cleanup(layout: HivePlotLayout, intra: bool = False)`

Nachbereitung der Pipeline, dient als Absicherung, dass Kanten und intra Knoten im Layout korrekt gesetzt sind.

`crossing_minimization.finish_structured_axis_orders(layout: HivePlotLayout, isolated_node_groups: dict[str, list[int]], expanded: bool = False) → None`

Finalisiert die Achsenordnungen nach der Barycenter-Pipeline.

Die Funktion fügt zunächst intra-axis Knoten und verbleibende Knoten pro Achse zusammen, hängt danach die zuvor entfernten isolierten Knoten wieder an und stellt sicher, dass intra-axis Kanten im Graphen rekonstruiert werden. Die endgültige Reihenfolge ist auf jeder Achse identisch: reine intra axis Knoten | mixed intra axis Knoten | inter axis Knoten (real) | isolierte Knoten | virtuelle Knoten (dummys).

Parameter

- **layout** – Das HivePlotLayout, dessen node_groups und Kanten finalisiert werden.
- **isolated_node_groups** – Pro Achse die während der Pipeline entfernten isolierten Knoten, die wieder angehängt werden sollen.

`crossing_minimization.intra_axis_handler(layout: HivePlotLayout) → None`

Bereitet intra-axis Knoten nach der Barycenter-Pipeline auf. Alle intra-axis Kanten werden zu Zusammenhangskomponenten gruppiert und pro Achse sortiert in zwei Kategorien abgelegt: kurze Pfade (Länge 2) und längere Pfade (Länge > 2). Diese Reihenfolge wird in `layout.intra_axis_nodes` gespeichert, um später wieder in die endgültige Achsenordnung integriert werden zu können. Es handelt sich hierbei um eine heuristische Sortierung der reinen intra axis Knoten, um beim Expandieren einer Achse die Knoten nach der Größe ihrer Zusammenhangskomponente zu ordnen.

Parameter

layout (`HivePlotLayout`) – Das HivePlotLayout mit gefüllten `intra_axis_nodes` und `intra_axis_edges` Feldern.

`crossing_minimization.parse_dummy_name(name: str) → tuple[int, int, int]`

Zerlegt den Namen eines Dummyknotens in seine Bestandteile: Startknoten, Endknoten und Sequenznummer. (siehe `make_dummy_name` Funktion für Namensschema)

Parameter

name (`str`) – Der zu zerlegende Dummyknoten

Rückgabe

Tupel aus int-Werten: (Startknoten, Endknoten, Sequenznummer)

Rückgabetyp

`tuple[int, int, int]`

`crossing_minimization.remove_isolated_nodes(graph: Graph, node_groups: dict[int, list[int]]) → dict[int, list[int]]`

Die Funktion entfernt alle isolierten Knoten aus der persistenten `node_group` des `HivePlotLayouts`.

Parameter

- **graph** (`nx.Graph`) – Der zugrundeliegende Graph aus dem `HivePlotLayout`.
- **node_groups** (`dict[str, list[int]]`) – Die persistente Knotenlisten der Achsen.

Rückgabe

Ein dict mit den isolierten Knoten pro Achse, key: Achse, value: Listen mit isolierten Knoten.

Rückgabetyp

`dict[str, list[int]]`

`crossing_minimization.subdivide_long_edges(layout: HivePlotLayout, node_position_map: dict[int | str, int], node_axis_map: dict[int | str, int], neighborhood_map: dict[int | str, list[int | str]], expanded: bool = False) → None`

Funktion dient dem Einfügen von Dummyknoten für lange Kanten (`span > 1`) auf den Achsen zwischen Start- und Endknoten. Schema: `d_[Startknoten]_[Endknoten]_[Sequenznummer]`: z.B. `d_5_10_2` ist der zweite Dummyknoten auf der zerlegten langen Kante von 5 nach 10. Die lange Kante kann dann folgendermaßen beschrieben werden: Startknoten - `d_5_10_1` - `d_5_10_2` - ... - `d_5_10_(span-1)` - Endknoten. Außerdem werden die inter axis Knoten (`Span = 0`) ermittelt und gefiltert, um später die Achsenreihenfolge sauber zu ermitteln und die Pipelineschritte zu Initialisieren. Gefiltert bedeutet hier: 1. nur intra axis die nicht mixed sind (vgl. `nested: is_mixed`, werden in 3b behandelt) 2. mixed (inter + intra, werden nur in 3a nicht aber in 3b berücksichtigt) Reine intra-axis Knoten werden aus den regulären `node_groups` entfernt und ihre Kanten vorübergehend aus dem Graphen gelöscht. Am Ende der Pipeline werden alle Informationen wieder konsistent zusammengesetzt.

Parameter

- **layout** ([HivePlotLayout](#)) – Das HivePlotLayout-Objekt, das die Informationen über den Graphen, die Achsen und die Knoten enthält.
- **node_position_map** (*dict[int | str, int]*) – Knoten-ID: Achsenindex in phi.
- **node_axis_map** (*dict[int | str, int]*) – Knoten-ID: Achse
- **(neighborhood_map** (*neighborhood_map*) – *dict[int | str, list[int | str]]*): Knoten-ID: Nachbarknoten (:= proper = Span = 1)
- **expanded** (*bool*) – dient der Unterscheidung, ob in der Pipeline mit expandierten Achsen gerechnet wird oder nicht, Default = False (nicht expandierter Fall)

Rückgabe

None. Die Funktion arbeitet in-place auf dem übergebenen Layout!

class `dblp_parser.LocalDTDResolver`(*dtd_path: str*)

Bases: `Resolver`

Da es beim Parse trotz schneller Hardware zu Performanzproblemen kam mussten Einstellungen feingranular festgelegt werden. Unter anderem in dieser Klasse gebündelt die dtd-Datei zur Übersetzung von Umlauten und Sonderzeichen. Sie wird, falls vorhanden, von lokal bevorzugt.

resolve(*self, system_url, public_id, context*)

Override this method to resolve an external source by `system_url` and `public_id`. The third argument is an opaque context object.

Return the result of one of the `resolve_*`(`)` methods.

`dblp_parser.build_node_identity_maps`(*nodes: list[str]*) → *tuple[dict[int, str], dict[str, int]]*

Funktion dient der Erstellung der Maps für die gleichnamigen Felder im Hiveplotlayout. Es werden KnotenIDs als Integer auf die Namen der Forschenden als Strings und vice versa gemappt. Dadurch wird es einerseits ermöglicht den Eingabegraphen in ein pipelinefreundliches Format zu übersetzen und gleichzeitig aus diesem am Ende die Label für den Renderer abzuleiten.

`dblp_parser.parse_gd_coauthor_networks`(*xml_path: str, cache_path: str = None*) → *dict[int, Graph]*

Parst dblp und gibt Co-Autoren-Graphen pro Jahr zurück. Ausgabe erfolgt als pickle, um die Daten wieder verwendbar zu halten, ohne den Parse wieder durchzuführen. Lädt aus Cache falls vorhanden.

`dblp_parser.scan_booktitles`(*xml_path: str, dtd_path: str*) → *set[str]*

Vorverarbeitung der dblp-xml Datei um nach booktitles zu filtern, da die xml-Datei neben Konferenzbänden auch andere wissenschaftliche Erzeugnisse wie Dissertationen oder Buchpublikationen enthält, die nichts mit der GD-Konferenz zu tun haben.

`debug_renderer.py` – Visuelle Debug-Hilfe für die Hiveplot-Pipeline.

Verwendung in beliebigen Modulen:

```
from src.debug_renderer import render_debug
```

```
render_debug(layout, title="nach subdivide", highlight_nodes=[...])
```

Die Funktion zeigt standardmäßig den Hiveplot-Achsen-Plot. Mit dem Flag `just_edges=True` können nur die Kanten aus dem Graphen gezeichnet werden.

`debug_renderer.render_debug`(*layout: HivePlotLayout, title: str = 'Debug', highlight_nodes: list | None = None, highlight_edges: list[tuple] | None = None, save_path: str | None = None, just_edges: bool = False*) → *None*

Rendert einen Hiveplot-Plot für das angegebene Layout.

Parameter

- **layout** – HivePlotLayout-Instanz.
- **title** – Titel des Plots.
- **highlight_nodes** – Liste von Knoten, die hervorgehoben werden sollen.

- **highlight_edges** – Liste von Kanten, die hervorgehoben werden sollen.
- **save_path** – Optionaler Pfad zum Speichern der Grafik.
- **just_edges** – Wenn True, werden nur die Kanten aus layout.graph gezeichnet (keine dummy_edge_segments oder intra_axis_edges).

`generator.generate_hiveplot(graph_mode: int, pipeline: str = 'bary')`

`generator.settings(svg_path, node_pt=5, line_pt=1.2, text_pt=0, draw_dummys=True, dummy_size=3)`

`graphs.sample_graph_caveman(num_cliques, clique_size)`

Erzeugt einen Caveman-Graphen. Diese Funktion dient dem Testen und Debugging.

Parameter

- **num_cliques** – Anzahl der Cliques
- **clique_size** – Knotenanzahl jeder Clique

Rückgabe

Caveman-Graph

Rückgabetyp

`nx.Graph`

`graphs.sample_graph_multipartite(sizes=(10, 10, 10))`

Erzeugt einen vollständigen multipartiten Testgraphen. Diese Funktion dient dem Testen und Debugging.

Parameter

sizes – Tuple mit Knotenanzahl pro Partition

Rückgabe

multipartiter Graph mit subset-Attribut

Rückgabetyp

`nx.Graph`

`graphs.sample_graph_selfconstructed()`

Erzeugt einen sehr kleinen, selbst konstruierten Graphen. Dient dem Testen und Debugging. Er weist folgende Kennzahlen auf: - 8 Knoten - 5 Kanten - 4 Achsen mit jeweils 2 Knoten -> $\Phi = (0, 1, 2, 3)$ - das optimale Gewicht ist 7, bei der Achsenanordnung (0, 1, 3, 2)

Rückgabe

selbst konstruierter Graph

Rückgabetyp

`nx.Graph`

`graphs.sample_graph_selfconstructed_extended(mode: int = 0)`

Verschiedene Testgraphen für Pipelinetests (Barycenter/ILP). Dient größtenteils für Proof of Concept bei Nischenfällen, deshalb während des Debuggens ständig verändert und erweitert.

Rückgabe

selbst konstruierter Graph

Rückgabetyp

`nx.Graph`

```
class hiveplot.HivePlotLayout(graph: ~networkx.classes.graph.Graph, num_axes: int, axis_order:  
    list[int], node_groups: dict[int, list[int]], node_groups_dummies:  
    dict[int, list[int]] = <factory>, dummy_edge_segments: list[tuple[int,  
    int]] = <factory>, long_edges: set[tuple[int, int]] = <factory>,  
    intra_axis_nodes: dict[int, list[int]] = <factory>, intra_axis_edges:  
    list[tuple[int, int]] = <factory>, node_order: dict[int, list[int]] =  
    <factory>, node_groups_expanded: dict[int, list[int | str]] = <factory>,  
    edges_expanded: list[tuple[int | str, int | str]] = <factory>, id_to_name:  
    dict[int, str] = <factory>, name_to_id: dict[str, int] = <factory>,  
    crossings: int | None = None, crossings_extended: int | None = None)
```

Bases: object

Zentrale Datenstruktur für ein berechnetes Hive-Plot-Layout.

graph

ein Simple Graph den man nach der Verarbeitung der geparsten DBLP Daten erhält: V = Autoren, E = Coautorenschaft zu

Type

nx.Graph

num_axes

k = Anzahl Achsen

Type

int

axis_order

phi: siehe native_order Funktion in ordering.py

Type

list[int]

node_groups

alpha: siehe node_groups Funktion in ordering.py

Type

dict[int, list[int]]

node_groups_dummies

Achsen mit Dummyknoten

Type

dict[int, list[int]]

dummy_edge_segments

key: Kantentupel (u,v) value: Liste von Kantensegmenten die u und v über Dummyknoten verbinden

Type

dict[tuple[int,int], list[tuple[int, int]]]

node_order

pi_i pro Achse gebündelt

Type

dict[int, list[int]]

node_order_plus

pi_i^+

Type

dict[int, list[int]]

node_order_minus

pi_i^-

Type

dict[int, list[int]]

crossings

Kreuzungszahl Standardmodell

Type

Optional[int]

crossings_extended

Kreuzungszahl erweitertes Modell

Type

Optional[int]

axis_order: list[int]**copy()**

Gibt eine Deepcopy des Hiveplotlayouts zurück.

crossings: int | None = None**crossings_extended: int | None = None****dummy_edge_segments: list[tuple[int, int]]****edges()** → list[tuple]

Gibt die Kantenliste des Graphen zurück. Notwendig, weil mit dem Networkx Edgeview nicht direkt gearbeitet werden kann.

Rückgabe

Liste der Kanten als Tupel (u, v)

Rückgabotyp

list[tuple]

edges_expanded: list[tuple[int | str, int | str]]**expand_axes**(node_axis_map: dict[int | str, int]) → None

Die Funktion dient der Umsetzung unterschiedlicher Knotenordnungen auf expandierten Achsen. Die Funktion ist ausschließlich zum pre-processing des Eingabegraphen bevor etwaige Berechnungen der Pipeline durchgeführt werden gedacht. Dazu wird die Achse i zu den Achsen mit KnotenIDs i und -i expandiert. Die intra-axis Kanten werden symmetrisch zwischen den Achsenkopien gezeichnet. Folgende Schritte werden durchgeführt: 1. Initialisieren der Map intra_expandables mit Key AchsenID einer Kante und ihrer intra-axis Kanten (Achse wird nur aufgenommen, wenn es intra-axis Kanten gibt) 2. Initialisieren der Map inter_expandables mit allen Kanten die genau einen Start oder Endknoten auf der expandierten Achse haben 3. Initialisieren der node_groups_expanded mit Keys ursprüngliche AchsenIDs und expandierte AchsenIDs und ihre in node_groups enthaltenen Knotenlisten (expandierte Achse -i bekommt alle Knoten von i zugeordnet, jedoch werden die KnotenIDs negativ gesetzt) 4. Update des Hiveplotlayouts mit neuer Achsenordnung und -anzahl 5. Initialisieren einer axis_position_map mit Key AchsenID und Value ist Position der Achse in der neuen Achsenordnung 6. Behandlung der intra-axis Kanten

- a. Entfernen der intra-axis Kanten aus dem Hiveplotlayout
- b. Erstellen der neuen intra-axis Kanten zwischen den expandierten Achsen (Kante (u,v) auf Achse i wird symmetrisch gespiegelt zu den Kanten (-u, v) und (u, -v))
- c. Einpflegen der neuen intra-axis Kanten in die Networkx Graphenstruktur des Hiveplotlayouts

7. Behandlung der inter-axis Kanten

- a. Entfernen der inter-axis Kanten aus dem Hiveplotlayout
 - b. **Erstellen der neuen inter-axis Kanten:**
 - I: Ziel- und Startknoten der Kante auf expandierter Achse: betrachte Kanten i/j mit Knoten u/v wenn i vor j in der Achsenordnung, Kante (u, v) zu (-u, v) andernfalls zu (u, -v) [i -> -i -> j -> -j]
 - II: Ziel- oder Startknoten der Kante auf expandierter Achse: betrachte Kanten i/j mit Knoten u/v (i expandiert), falls i vor j in der Achsenordnung (u,v) zu (-u, v), andernfalls Kante übernehmen [j -> i -> -i]
 - c. Einpflegen der neuen inter-axis Kanten in die Networkx Graphenstruktur des Hiveplotlayouts
8. Einpflegen und Zuordnung zu Subsets der neuen Knoten in der Networkx Graphenstruktur des Hiveplotlayouts

Parameter

node_axis_map(*dict[int | str, int]*) – Knoten-ID: Achsen-ID

fuse_edges_with_edge_dummies(*expanded: bool = False*) → list[tuple[int, int]]

Erzeugt eine Liste die alle kurzen Kanten und Dummykanten vereinigt zurückgibt.

Rückgabe

Vereinigung aus Kanten und Dummykanten

Rückgabetyt

list[tuple[int, int]]

fuse_node_groups_with_dummies(*expanded: bool = False*) → dict[int, list[int]]

Erzeugt ein dict mit Achsen als Schlüssel und der vereinigten Menge aus Knoten und Dummyknoten pro Achse als Wert. Zuerst die realen dann die virtuellen Knoten. :param expanded: dient der Unterscheidung, ob in der Pipeline mit expandierten Achsen gerechnet wird oder nicht, Default = False (nicht expandierter Fall) :type expanded: bool

Rückgabe

Vereinigung der Knoten und Dummyknoten pro Achse.

Rückgabetyt

dict[int, list[int]]

get_proper_neighborhood_map(*fused_edge_list: list[tuple[int | str, int | str]]*, *expanded: bool = False*) → dict[slice(int | str, list[int | str], None)]

Die Funktion ermittelt eine Liste, die jeden Knoten (sowohl real als auch virtuell) auf eine Liste seiner Nachbarn mappt.

Parameter

- **fused_edge_list** – list[tuple[int | str, int | str]]: Liste der realen und virtuellen Kantentupel
- **expanded** (*bool*) – dient der Unterscheidung, ob in der Pipeline mit expandierten Achsen gerechnet wird oder nicht, Default = False (nicht expandierter Fall)

Rückgabe

list[int | str]: KnotenID: Nachbarliste

Rückgabetyt

dict[int | str

graph: Graph

id_to_name: dict[int, str]

intra_axis_edges: list[tuple[int, int]]

intra_axis_nodes: dict[int, list[int]]

```

long_edges: set[tuple[int, int]]
name_to_id: dict[str, int]
node_groups: dict[int, list[int]]
node_groups_dummies: dict[int, list[int]]
node_groups_expanded: dict[int, list[int | str]]
node_order: dict[int, list[int]]
nodes_on_axis(axis: int, include_dummies: bool = False) → list[int]

```

Gibt die Knoten einer bestimmten Achse zurück.

Parameter

- **axis** (*int*) – Die Achse, für die die Knoten zurückgegeben werden sollen
- **include_dummies** (*bool*) – false (default) Rückgabe ohne dummies, true mit dummies

Rückgabe

Liste der Knoten auf der angegebenen Achse

Rückgabetypp

list[int]

num_axes: int

post_processing_expansion(*node_axis_map: dict[int | str, int], dummy_copy: list[tuple[int | str, int | str]] = None*) → None

Die Funktion dient dem post-processing des Eingabegraphen nach der Pipeline, sodass bei diesem die Achsen expandiert werden. Dazu wird die Achse i (> 0) zu den Achsen mit KnotenIDs i und $-i$ expandiert. Die intra-axis Kanten werden symmetrisch zwischen den Achsenkopien gezeichnet. Folgende Schritte werden durchgeführt: 1. Initialisieren der Dummysegmente, diese werden in der Funktion multipliziert 2. Initialisieren der Map `intra_expandables` mit Key AchsenID einer Kante und ihrer intra-axis Kanten (Achse wird nur aufgenommen, wenn es intra-axis Kanten gibt) 3. Initialisieren der Map `inter_expandables` mit allen Kanten die genau einen Start oder Endknoten auf der expandierten Achse haben 4. Initialisieren der `node_groups_expanded` mit Keys ursprüngliche AchsenIDs und expandierte AchsenIDs und ihre in `node_groups` enthaltenen Knotenlisten (expandierte Achse $-i$ bekommt alle Knoten von i zugeordnet, jedoch werden die KnotenIDs negativ gesetzt, für virtuelle Knoten wird die Sequenznummer inkrementiert und die neue Kante zwischen den virtuellen Knoten direkt in das Networkx Graphmodell hinzugefügt) 5. Update des Hiveplotlayouts mit neuer Achsenordnung und -anzahl und Update der `edge_axis_map` 6. Initialisieren einer `axis_position_map` mit Key AchsenID und Value Position der Achse in der neuen Achsenordnung 7. Behandlung der intra-axis Kanten

- Entfernen der intra-axis Kanten aus dem Hiveplotlayout
- Erstellen der neuen intra-axis Kanten zwischen den expandierten Achsen (Kante (u,v) auf Achse i wird symmetrisch gespiegelt zu den Kanten $(-u, v)$ und $(u, -v)$)
- Einpflegen der neuen intra-axis Kanten in die Networkx Graphenstruktur des Hiveplotlayouts

8. Behandlung der inter-axis Kanten

- Entfernen der inter-axis Kanten aus dem Hiveplotlayout und den Dummykantensegmenten
- Erstellen der neuen inter-axis Kanten:**
 - Ziel- und Startknoten der Kante auf expandierter Achse: betrachte Kanten i/j mit Knoten u/v wenn i vor j in der Achsenordnung, Kante (u, v) zu $(-u, v)$ andernfalls zu $(u, -v)$. Handelt es sich um virtuelle Knoten wird die ID nicht negativ gesetzt sondern die Sequenznummer inkrementiert. II: Startknoten der Kante auf expandierter Achse: betrachte Kanten i/j mit Knoten u/v (i expandiert)

i.) falls j vor i in der Achsenordnung: Kante übernehmen ii.) falls i vor j in der Achsenordnung: (u,v) zu (-u, v), andernfalls Kante übernehmen, bei virtuellen Knoten wird die Sequenznummer inkrementiert statt die Knoten-ID negativ zu setzen

III: Zielknoten auf expandierter Achse: Behandlung wie in II, jedoch mit vertauschten Knoten i und j

c. Einpflegen der neuen inter-axis Kanten in die Networkx Graphenstruktur des Hiveplotlayouts

Parameter

- **node_axis_map** (*dict[int | str, int]*) – Knoten-ID: Achsen-ID
- **dummy_copy** (*list[tuple[int | str, int | str]]*, *optional*) – eine Aktuelle Kopie der Dummysegmente, Default ist None

updated_nodes (*fused_node_groups*) → *list[int]*

Gibt die Knotenliste des Graphen zurück. Notwendig, weil die Knoten sich ändern und je nach Pipeschritt die Knoten geupdated werden müssen.

Parameter

fused_node_groups (*dict[int, list[int | str]]*) – AchsenID: Knotenliste (real und virtuell)

Rückgabe

Liste der realen und virtuellen Knoten

Rückgabetyt

list[int]

ip_model.delta_mapping (*fused_groups: dict[int | str, list[int | str]]*) → *dict[tuple[int | str, ...], 1]*

Dient der Initialisierung der Ordnungsvariablen ($\delta_{i,u,v}$) für das 1L2S-ILP. Erzeugt aus der Vereinigung von virtuellen und realen Knoten das Mapping.

Parameter

layout ([HivePlotLayout](#)) – das zugrundeliegende Hiveplotlayout

Rückgabe

key: (u, v, $\alpha(u,v)$), value: None

Rückgabetyt

dict[tuple[int | str, ...], None]

ip_model.induced_crossings (*delta_static: dict[tuple[int | str, int | str, int | str], int]*, *sorted_neighbors: dict[int, list[int | str]]*, *pi_fix: int*, *pi_var_id: int*, *pi_var: list[int | str]*) → *lpSum*

Bildet die $C(\pi_i, \pi^{\pm})$ Zielfunktion nach. Die Funktion arbeitet folgendermaßen: 1. Betrachte paarweise Knoten u, v auf variabler Achse und ermittle die Ordnungsvariablen uv und vu 2. bestimme alle Nachbarn s von u und t von v auf der fixierten Achse und iteriere über alle Paare (s, t) mit $s \neq t$ 3. bestimme die Ordnungsvariablen st und ts 4. Füge den Kreuzungsterm $\delta_{uv} * \delta_{ts} + \delta_{vu} * \delta_{st}$ zur Zielfunktion hinzu 5. Rückgabe der Summe aller Kreuzungsterme als Pulp-Ausdruck

Parameter

- **delta_static** (*dict[tuple[int | str, int | str, int | str], int]*) – ordnungsvariable: belegung
- **sorted_neighbors** (*dict[int, list[int | str]]*) – ($\pi^{\pm}$ /ID, node aus π_i): Liste der Nachbarn von node auf der jeweiligen Achse
- **pi_fix** (*int*) – AchsenID der fixierten Nachbarachse
- **pi_var_id** (*int*) – AchsenID der variablen Achse
- **pi_var** (*int*) – Liste der Knoten auf der variablen Achse

Rückgabe

Anzahl der induzierten Kreuzungen zwischen `pi_var` und `pi_fix`

Rückgabetypp

`pp.lpSum`

`ip_model.ip_model_pipeline(layout: HivePlotLayout, threshold: int = 10, expanded: bool = False) → None`

Die Funktion führt die gesamte Pipeline zur Kreuzungsminimierung durch, inklusive der Berechnung des 1L2S-ILP. Alle Schritte werden in-place am `HivePlotLayout` durchgeführt. Ablauf expandiert: 1. anfängliches Layout expandieren 2. isolierte Knoten entfernen 3. lange Kanten unterteilen und Dummyknoten einfügen (virtuelle Knoten) 4. 1L2S-ILP berechnen 5. Herstellen der Achsenordnung und letztes Update der Datenstrukturen um die Visualisierung starten zu können.

Ablauf nicht expandiert: 1. isolierte Knoten entfernen 2. lange Kanten unterteilen und Dummyknoten einfügen (virtuelle Knoten) 3. 1L2S-ILP berechnen 4. Behandlung der intra-axis Kreuzungen 5. Herstellen der Achsenordnung und letztes Update der Datenstrukturen um die Visualisierung starten zu können.

Parameter

- **layout** (`HivePlotLayout`) – das zugrundeliegende Hiveplotlayout
- **threshold** (`int`, *optional*) – Anzahl der Sweeps bis zum Abbruch, Defaultwert ist 10.
- **expanded** (`bool`) – dient der Unterscheidung, ob in der Pipeline mit expandierten Achsen gerechnet wird oder nicht, Default = False (nicht expandierter Fall)

`ip_model.onelayer_twosided_optimization(layout: HivePlotLayout, neighborhood_map: dict[str | int, list[int | str]], node_axis_map: dict[str | int, int], threshold: int = 10, expanded: bool = False) → None`

Enthält die gesamte Logik, um das One Layer Two Sided Integer Linear Program (1L2S ILP) zu definieren und zu berechnen. Die Funktion verändert das `HiveplotLayout` in-place. Die Pipeline besteht aus Sweeps. Ein Sweep umfasst eine clockwise und eine counterclockwise Berechnung über jede Achse des Hiveplots. Folgende Schritte werden in der Funktion durchgeführt: 1. Initialisierung der Ordnungsvariablen ($\delta^i_{u,v}$) durch `delta_mapping()` 2. Initialisierung der Sweepschleife, Abbruch bei Erreichen eines Thresholds 3. Danach schließt sich ein Sweep an erst in clockwise dann counterclockwise Richtung, beide haben das gleiche Schema:

- Erstellen einer Problemistanz und einer Kopie für die aktuelle Achse
 - Initialisierung der Ordnungsvariablen für die aktuelle Achse (virtuelle und reale Knotenordnung werden in einem Schritt berechnet, aber die Achsenordnung `real < virtuell` muss berücksichtigt werden, da Gaps nicht explizit behandelt werden)**
 - Ordnungsvariablen getrennt für reale und virtuelle Achsenabschnitte
 - Ordnungsvariablen für `real < virtuell` = konstant 1
 - Ordnungsvariablen für `virtuell < real` = konstant 0
 - Erstellen der Zielfunktion speziell für die variable Achse
 - Erstellen der Nebenbedingungen für:**
 - Antisymmetrie (im Paper implizit behandelt)
 - Transitivität
 - Lösen des ILP
 - Synchronisieren der Lösung mit `delta`
4. Aktualisieren des Layouts durch Übersetzung von `delta` zurück in die Achsenordnung, da die variable Achse auf einem geupdateten Stand ermittelt werden muss, wird dies am Ende eines Sweeps durchgeführt

Parameter

- **layout** (`HivePlotLayout`) – `_description` das zugrundeliegende `HivePlotLayout`
- **neighborhood_map** (`dict[str | int, list[int | str]]`) – KnotenID: Liste von proper Nachbarn

- **node_axis_map** (*dict[str | int, int]*) – KnotenID: Achse
- **threshold** (*int, optional*) – Anzahl der Sweeps (=1x cw+ 1x ccw), Defaultwert ist 10
- **expanded** (*bool*) – dient der Unterscheidung, ob in der Pipeline mit expandierten Achsen gerechnet wird oder nicht, Default = False (nicht expandierter Fall)

`logger_setup.setup_logger(per_session=True)`

Standardsetup für Logging.

`main.main()`

`ordering.brute_force_ordering(ordering: list[int], node_grps: dict, edges: list) → tuple[int, ...]`

Für kleine k und zum Entwickeln der Pipeline vermeide ich zuerst die Auseinandersetzung mit dem Solver. Diese Funktion dient dem schnellen Berechnen von Pipelineschritt 2: günstigste Achsenanordnung finden.

Parameter

- **ordering** (*list[int]*) – aktuelle Achsenordnung (phi)
- **node_grps** (*dict*) – key: Achse, value: Knoten
- **edges** (*list*) – Knotenliste

Rückgabe

`_description_`

Rückgabetyyp

`tuple[int, ...]`

`ordering.native_order(nodes)`

Bekommt einen NodeView übergeben und wandelt diese über ein Set in eine Liste um, die alle Subsetnummern enthält. Diese Subsets spiegeln die Achsen wieder. Die Liste dient als Initialisierung zur Berechnung von Phi. Es handelt sich als o

Parameter

nodes (*NodeDataView*) – siehe `networkx.graph.nodes`

Rückgabe

zyklische Achsenanordnung mit Achsen-IDs

Rückgabetyyp

`list`

`ordering.node_groups(nodes: list[tuple[int, int]])`

Erzeugt ein Dict wobei jeder Key ein Subset ist dem eine Liste aus zugehörigen Knoten zugeordnet wird. Es handelt sich hierbei um die Funktion alpha.

Parameter

nodes (*NodeView*) – Liste aus Tupeln (KnotenID, Subset)

Rückgabe

key: int, value: list {0: [0, 1, 2, 3...], ... }

Rückgabetyyp

`dict`

`ordering.node_to_axis(node, node_grps, position=False)`

Gibt an auf welcher Achse sich ein Knoten befindet. Funktion wird nur zum Initialisieren verwendet. Für Iterationen in der Pipeline ist sie nicht performant. Dafür werden Mappings benutzt (z.B. siehe `ordering.node_to_axis_maps`)

Parameter

- **node** (*int*) – Knoten ID
- **node_grps** (*dict*) – key: Achse, value: Knoten

- **position** (*bool*) – Rückgabe der Achse, true Rückgabe der Position der Achse in der Achsenordnung (*phi*). False ist default.

Verursacht

ValueError – Der Knoten kann keiner Achse zugeordnet werden.

Rückgabe

Achsen ID

Rückgabetyt

int

`ordering.node_to_axis_maps(layout: HivePlotLayout, fused_node_groups: dict[int, list[int | str]]) → tuple[dict[int | str, int], dict[int | str, int]]`

Die Funktion berechnet zwei Maps um für einen gegebenen Knoten sehr schnell 1. $\alpha(\text{Knoten})$ und 2. $\phi[\alpha(\text{Knoten})]$ zu bestimmen.

Parameter

- **layout** (*HivePlotLayout*) – Das Hiveplotlayout
- **fused_node_groups** (*dict[int, list[int | str]]*) – Achse: Knotenliste (virtuell und real)

Rückgabe

Knoten: Achse & Knoten: Achsenposition in Φ

Rückgabetyt

`tuple[dict[int | str, int], dict[int | str, int]]`

`ordering.reordered_node_groups(node_grps: dict[int, list[int]], new_order: tuple[int, ...]) → dict[int, list[int]]`

Schnelles Umordnen nach Optimierung der Achsenordnung.

`partitioning.cluset_newman_moore_communities(graph, threshold: int = 0)`

Berechnet die Communities eines Graphen per Clauset-Newman-Moore Algorithmus und wandelt die frozen-set Liste in ein dict um. Falls ein Threshold übergeben wird, so wird die Anzahl der Communities auf den Threshold gestaucht. So wird sichergestellt, dass man eine Anzahl Communities (Achsen) erhält, die noch einen übersichtlichen Hiveplot erzeugen lassen.

Parameter

- **graph** (*nx.Graph*) – Der Eingabegraph für den die Communities berechnet werden sollen.
- **threshold** (*int*) – maximale Anzahl Communities, bei Default (=0) wird die natürliche Anzahl von Communities ermittelt und zurückgegeben

Rückgabe

Ein Dictionary mit den Communities als Werte und ihren IDs als Schlüssel.

Rückgabetyt

`dict[int, list[int]]`

`partitioning.louvain_community_detection(graph)`

Berechnet die Communities eines Graphen per Louvain Community Detection und wandelt die frozen-set Liste in ein dict um.

Parameter

graph (*nx.Graph*) – Der Eingabegraph, für den die Communities berechnet werden sollen.

Rückgabe

Ein Dictionary mit den Communities als Werte und ihren IDs als Schlüssel.

Rückgabetyt

`dict[int, list[int]]`

renderer.draw_basis(*node_groups*: dict[int, list[str | int]], *edges*: list[tuple[int | str, int | str]], *intra_edges*: list[tuple[int | str, int | str]] | None = None, *axes_labels*: bool = True) → tuple[list[str], list[str], list[str], list[str]]

Die Funktion berechnet die Koordinaten für Achsen, Kanten, Knoten und deren Label und erzeugt die .svg-
Zeilen. Einige Hinweise zu den Berechnungen: 1. angle = 270 + i * (360/len(node_groups)): 270 Grad, sorgt
dafür, dass die erste Achse der Ordnung immer Richtung Norden eingefügt wird und alle weiteren radial dazu
im Uhrzeigersinn 2. radius bei realen Knoten = offset vom Ursprung + gleichmäßige Anordnung der Knoten
auf der Achse, wobei die Knoten nie am Achsenanfang oder -ende beginnen 3. x Wert für Labels: {x + r + 2}
x Koordinate des zugehörigen Knoten + Radius des Knoten + Offset zum Knoten (verhindert das Überlappen
von Label und Knoten) 4. radius bei virtuellen Knoten = Achsenendpunkt + gleichmäßige Anordnung der
Knoten auf einer imaginären Achse, wobei sie einen Abstand zum Startpunkt haben (Länge der Imaginären
auf 50 Pixel begrenzt) 5. x_fix, y_fix bei nicht intra Kanten: Fixpunkt zu dem die Kante tendiert, wird auf
den Mittelpunkt der Kante + ein Offset alpha (unterschiedlich für real-real/virtuell-virtuell und real-virtuell)
vom Koordinatenursprung weg gesetzt 6. x_fix, y_fix bei intra Kanten: da Start- und Endknoten auf der
gleichen Achse liegen wird der Mittelpunkt nicht wie im vorherigen Fall entgegen des Ursprungs versetzt
von der Achsen weg anhand des Offset alpha

Parameter

- **node_groups** (dict[int, list[str | int]]) – AchsenID: Knotenliste
- **edges** (list[tuple[]]) – gesamte Kantenliste
- **intra_edges** (list[tuple[, optional)]) – Liste der intra-Kanten, Default ist None

Rückgabe

svg strings für die Graphenelemente

Rückgabebetyp

tuple[list[str], list[str], list[str], list[str]]

renderer.hiveplot_renderer(*name*: str, *layout*: HivePlotLayout, *expanded*: bool = False, *intra*: bool = False, *mode*: str = 'debug', *node_labels*: bool = True, *axes_labels*: bool = True) → None

Pipeline die das Rendern des fertig berechneten Hiveplotlayouts in eine Scalable Vector Graphics (.svg) realisiert.

renderer.render_svg(*filename*: str, *width*: int, *height*: int, *elements*: list[str]) → None

Setzt den Header der .svg Datei und hängt die Graphenelemente an und setzt die Schlussklausel. Datei wird neu erstellt bzw. überschrieben, falls sie bereits existiert.

renderer.translate_polar_to_carthesian(*radius*: float, *angle*: float, *center_x*: float = 500.0, *center_y*: float = 500.0) → tuple[float, float]

Umrechnung der Polarkoordinate in ein karthesisches Tupel.

C

cost, ??
crossing_minimization, 1

d

dblp_parser, 4
debug_renderer, 4

g

generator, 5
graphs, 5

h

hiveplot, 5

i

ip_model, 10

l

logger_setup, 12

m

main, 12

o

ordering, 12

p

partitioning, 13

r

renderer, 13